

Project Notes: Duolingo-style English Learning Homepage

Audience: CSE students — architecture, file layout, coding patterns, and how to run/extend the project.

1. High-level overview

- This workspace contains a small multi-page web project plus a sub-app `cosmic-planet-course-map` (a Vite + React + TypeScript single-page app) under `uploads/cosmic-planet-course-map/src`.
- The top-level HTML files (e.g., *Duolingo Style Homepage.dc.html*, *Course Dashboard.dc.html*) are page templates and static views using plain HTML/CSS/JS.
- Shared scripts and data are in *curriculum.js*, *parent-link.js*, *support.js* and other sibling JS files.

2. Files & directories to know

- **Duolingo Style Homepage.dc.html** — main landing page. Contains markup and references to local assets.
- **curriculum.js** — likely contains course data and structures used by multiple pages.
- **parent-link.js**, **support.js** — helper scripts for navigation, analytics, or UI helpers.
- **uploads/cosmic-planet-course-map/** — standalone Vite app (React + TypeScript):
 - *package.json* — npm scripts and dependencies.
 - *tsconfig.json*, *vite.config.ts* — TypeScript + dev server config.
 - *src/* — app source: *App.tsx*, *main.tsx*, *components/*, *data/*, *lib/*.
 - *src/components/* — modular UI components (e.g., *CourseMap.tsx*, *PlanetNode.tsx*).

3. How the code is organized and written

- Plain HTML pages: Structure content with semantic HTML; small inline or external scripts handle page-level behaviors.
- Vanilla JS helpers: *curriculum.js* and *support.js* appear to export or attach global objects to manage data and interactions across multiple HTML pages.
- React app (*cosmic-planet-course-map*): Uses TypeScript + React functional components.
- *App.tsx* glues the app and routes (if any).
- *components/* contain single-responsibility components; follow the pattern: presentational + small stateful parents.
- *lib/sound.ts* centralizes audio/sfx logic.

4. Coding conventions & best practices used (and suggested)

- Use descriptive component/file names: *PlanetNode.tsx* is specific to the UI element.
- Keep components small: Each component should have one responsibility (render node, modal, header, etc.).

- Type safety: *types.ts* defines domain types — prefer explicit types for props and state in TSX files.
- Centralize side-effects: Keep audio, localStorage, or network code in dedicated helper modules (*lib/*).
- Assets: Place images, sounds, and static JSON under *assets/* and reference them by relative URLs.
- Avoid global mutable state in page scripts; prefer module-scoped state or simple stores.

5. Data flow and runtime behavior

- Static pages load *curriculum.js* or other scripts that attach data (global or module exports) to the window, then render UI using DOM APIs.
- React sub-app imports *courses.ts* and renders UI declaratively. Props flow from parent to child; components use local `useState` for ephemeral UI state.
- Events (clicks, modal open/close) are handled by component handlers or small utilities.

6. How to run the React app locally

```
cd "uploads/cosmic-planet-course-map"
npm install
npm run dev
```

Open the printed local URL (Vite prints it, usually *http://localhost:5173*).

7. How to build a production bundle (React app)

```
cd "uploads/cosmic-planet-course-map"
npm run build
# output will be in dist/
```

8. How to extend or contribute (practical tips)

- When adding new components, add a new file under *src/components/* and export it as default. Keep props typed and minimal.
- Add unit tests for pure logic modules (e.g., *lib/sound.ts*) using a test runner like Vitest or Jest.
- Document data shape in *types.ts* and update *src/data/courses.ts* when course content changes.
- For global scripts (non-React pages), avoid polluting *window* — export a function and call it from inline script tags.

9. Common tasks a CSE student might do

- Add a new lesson node: modify *src/data/courses.ts* and add a matching *PlanetNode* rendering case.
- Add scoring or persistence: create a small storage module that wraps `localStorage` and exports typed getters/setters.
- Add accessibility: ensure interactive elements use `button` elements, provide `aria-*` attributes, and keyboard handlers.

10. Converting these notes to PDF

- Open this HTML in a browser and Print → Save as PDF.
- Or use `pandoc` to convert the Markdown file to PDF:

```
pandoc docs\project-notes.md -o docs\project-notes.pdf
```

11. Next steps and suggestions

- Add a *README.md* at repo root with run/build instructions and a link to *docs/project-notes.md*.
- Add a small *CONTRIBUTING.md* describing coding style and PR process.
- Add basic tests and an npm *preview* script for static pages.

Generated for quick reference; ask me to (1) convert this to PDF here if you want me to attempt it, (2) add a README, or (3) extract a developer quickstart script.